

ReplayScope: Testing Smart Contract Signature Replay

CSE 5472 | Project Proposal | Team review draft

Team members. [Insert all team members' full names before submission.]

Project and threat model. We will build a configurable testing tool for EVM smart contracts that use EIP-712 signatures to authorize one-time actions. An attacker possesses a previously valid signature, but not the signer's private key, and attempts to reuse it in the same contract or a different contract. Our tool will generate valid signed requests, construct replay sequences, and check whether execution causes an unauthorized state change. Testing will run on a local EVM with test keys and assets. The initial scope is signed withdrawals and permit-style token approvals; cross-chain replay is a stretch goal.

Related work and limitations. Representative established approaches include Slither's static analysis and Echidna's property-based fuzzing [1, 2]. These provide useful analysis infrastructure, but applying them to application-specific authorization requires appropriate models or properties. EIP-712 defines structured signing and domain separation but explicitly excludes replay protection; ERC-2612 specifies nonce-based protection for signed token approvals [3, 4]. Our focus is executable testing of intended signature-use restrictions. We do not assume that existing tools cannot detect these bugs; their coverage will be measured on our benchmark.

Contribution and security impact. We will implement a Python command-line tool that generates Solidity/Foundry test harnesses from adapters describing signature fields, entry points, and expected state changes. New code will automate signature reuse, deployment of a second contract instance, and assertion generation. Reports will include a reproducible failing sequence. For approvals, tests will attempt to restore spending authority after it has been consumed. Repeated calls with no unauthorized effect will not count as vulnerabilities. If successful, this workflow will help developers catch authorization failures before deployment and retain regression tests after fixes.

Mid-semester milestone. We will demonstrate the complete workflow on at least three vulnerable/fixed pairs, covering both replay categories. The update will include the threat model, adapter format, generated tests, initial detection and runtime results, and a live demonstration in which a vulnerable contract fails and its repaired counterpart passes. This establishes an operational prototype and evaluation pipeline before expanding coverage.

Final artifact and evaluation. The final repository will contain the CLI, adapters, generated tests, labeled benchmark, and reproducible evaluation scripts. We target 20-30 cases, including secure controls and documented bug/fix variants, with provenance recorded. We will reserve contract families for held-out evaluation and report constructed and independently sourced cases separately. We will compare against unmodified Slither and a Foundry fuzzing baseline using the same adapters and state-change assertions but without targeted replay generation. Under fixed time budgets, we will measure recall, false-positive rate, runtime, and successful exploit reproduction; unsupported cases will be reported separately. Randomized runs will use five seeds.

Completion risks and mitigation. Signature formats and state semantics may require manual adapters, while public labeled examples may be scarce. We will prioritize two documented templates and a small reproducible benchmark, adding public cases as available. We will report manual setup effort and limit conclusions to supported contracts. If integration takes longer than expected, we will reduce template coverage while retaining both replay categories, secure controls, and the planned comparisons.

[1] Slither: Static Analyzer for Solidity and Vyper.

[2] Echidna: Ethereum Smart Contract Fuzzer.

[3] EIP-712: Typed Structured Data Hashing and Signing.

[4] ERC-2612: Permit Extension for EIP-20 Signed Approvals.